Reverse Engineering of a Real-Time Chat Application

Presented By-*Ritik Kumar Shahi*
Roll No-*23BCS110*
Course-*Software Design & Architecture*
Professor-*Vivekraj VK*

What is Reverse Engineering?

- **Definition-** Reverse engineering is the process of analyzing an existing software system to recover its design, architecture, and requirements from the source code or executable program.
- **Purpose-** It helps developers understand how a system works when documentation or source code is missing or unclear, making maintenance and updates easier.
- **Key Applications-** Reverse engineering is used for security analysis, malware investigation, recovering lost source code, and understanding complex systems.
- **Benefits-**It can reduce development costs, simplify complex software, and enable integration or customization of existing systems.
- **Outcome-**The process produces higher-level representations such as documentation, UML diagrams, or system models to better understand the software structure.

About the Chat Application...

Overview

- A real-time chat application supporting one-on-one messaging and group chats.
- Built with the MERN Stack: MongoDB, Express.js, React.js, and Node.js.
- Real-time communication powered by Socket.IO (WebSockets).
- User authentication using JWT (JSON Web Tokens) and password hashing with bcryptjs.
- State management using Redux Toolkit.
- Responsive UI styled with Tailwind CSS.

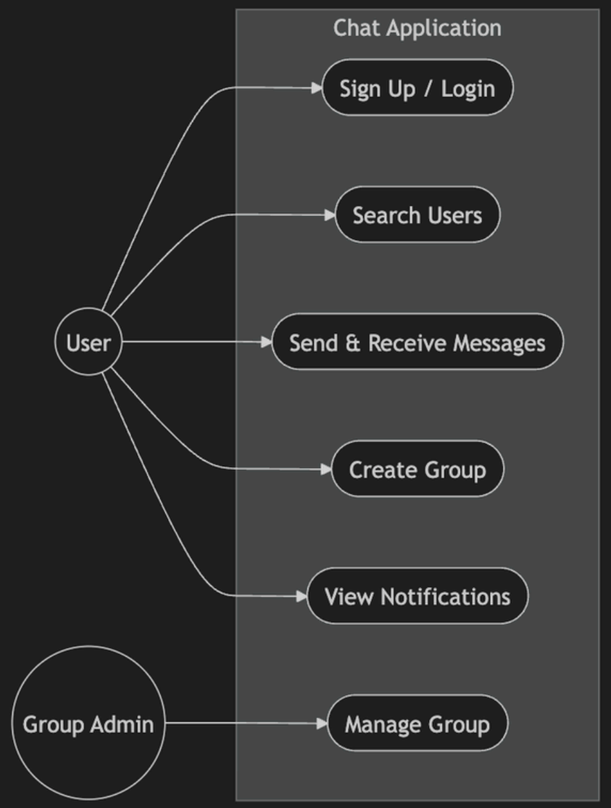
Key Features:

- User Registration & Login
- Real-time messaging with typing indicators
- Group chat creation, rename, add/remove members
- Message notifications with sound alerts
- Chat clearing and deletion

Use Case

#	Use Case	Actor	Description
UC1	Register Account	User	Sign up with firstName, lastName, email, password
UC2	Login	User	Authenticate with email & password, receive JWT
UC3	View Profile	User	View own profile details and avatar
UC4	Search Users	User	Search all registered users to start a chat
UC5	Start 1-on-1 Chat	User	Create or open a direct chat with another user
UC6	Send Message	User	Type and send a text message in a chat
UC7	Receive Message	User, System	Receive messages in real-time via Socket.IO
UC8	View Chat History	User	Load and view all previous messages in a chat
UC9	Clear Chat	User	Delete all messages in a chat
UC10	Create Group Chat	User	Create a group with 2+ users, become admin
UC11	Rename Group	Group Admin	Change the group chat name
UC12	Add Member	Group Admin	Add a user to the group
UC13	Remove Member	Group Admin	Remove a user from the group
UC14	Delete Group	Group Admin	Delete group and all its messages
UC15	View Notifications	User	See unread message notifications
UC16	Typing Indicator	User, System	See when another user is typing
UC17	Logout	User	Sign out and clear session

Minimilastic Idea

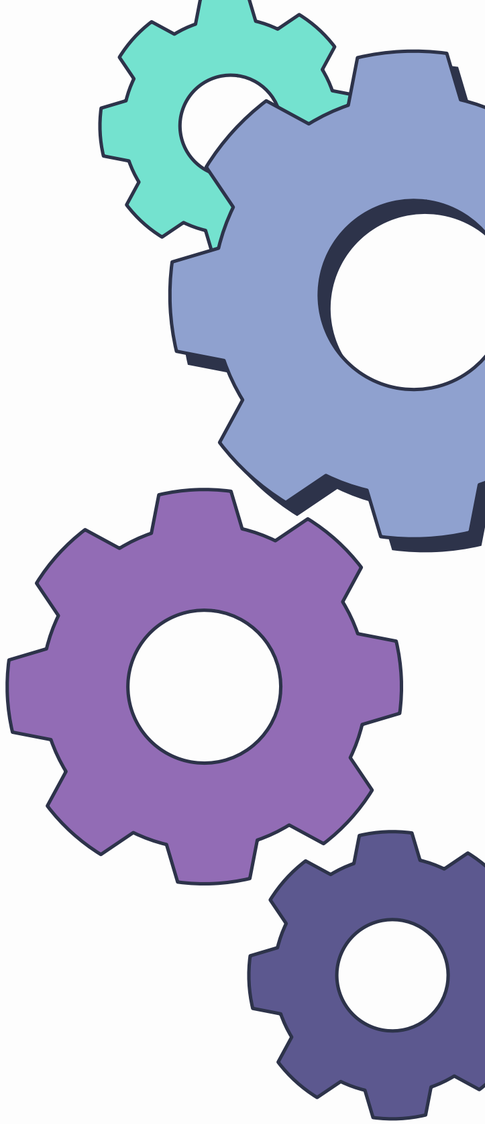


Use Case	What It Covers
Sign Up / Login	Register, login, logout, view profile
Search Users	Find users, start 1-on-1 chat
Send & Receive Messages	Send, receive (real-time), view history, clear chat, typing indicator
Create Group	Create group chat with 2+ members
Manage Group	Rename, add/remove members, delete group *(Admin only)*
View Notifications	See unread message alerts

High Level Architecture

System Architecture

- The system follows a Client-Server architecture with an additional real-time WebSocket layer.
- The React frontend communicates with the Express backend through REST API calls over HTTP for CRUD operations.
- For real-time features (messaging, typing indicators, notifications), the frontend connects to the Socket.IO server via WebSockets.
- The backend connects to MongoDB using Mongoose ODM for all data persistence.
- JWT tokens secure all protected API endpoints through an authorization middleware.



Database Design: ER Diagram

The application uses **3 MongoDB collections**:
-**User**:Stores user credentials and profile info.
-**Chat**:Represents a conversation (1-on-1 or group).
-**Message**:Stores individual messages.

Relationships:
- A User can participate in many Chats (Many-to-Many).
- A Chat contains many Messages (One-to-Many).
- A Message is sent by one User (Many-to-One).
- A Chat has one optional groupAdmin (for group chats).

Class Diagram – Backend Modules

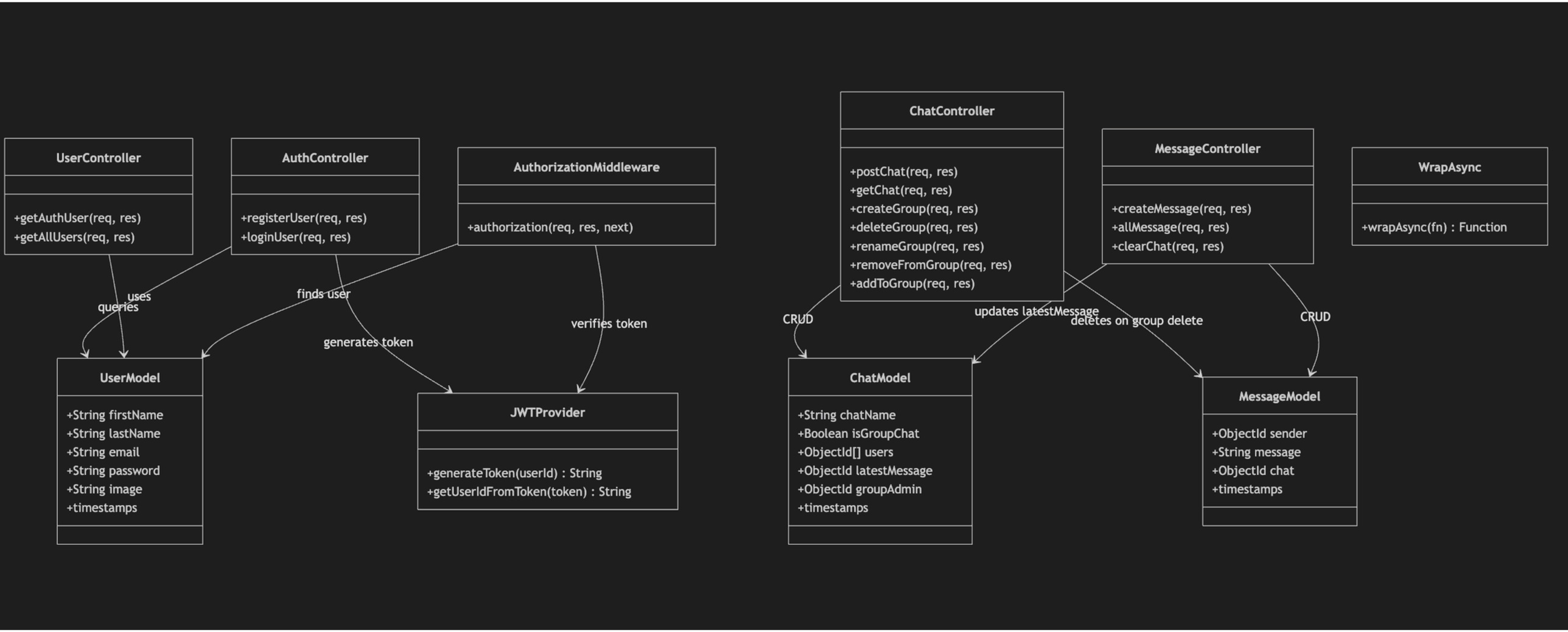
The backend is organized into 5 module categories:

1. Models (3): `UserModel`, `ChatModel`, `MessageModel` (defines MongoDB schemas).
2. Controllers (4): `AuthController`, `UserController`, `ChatController`, `MessageController` (Contain business logic).
3. Routes (4): `authRouter`, `userRouter`, `chatRouter`, `messageRouter` – (maps URLs to controllers).
4. Middlewares (2): `AuthorizationMiddleware` (JWT verification), `WrapAsync` (async error handler).
5. Config (1): `JWTProvider` (token generation and verification utility).

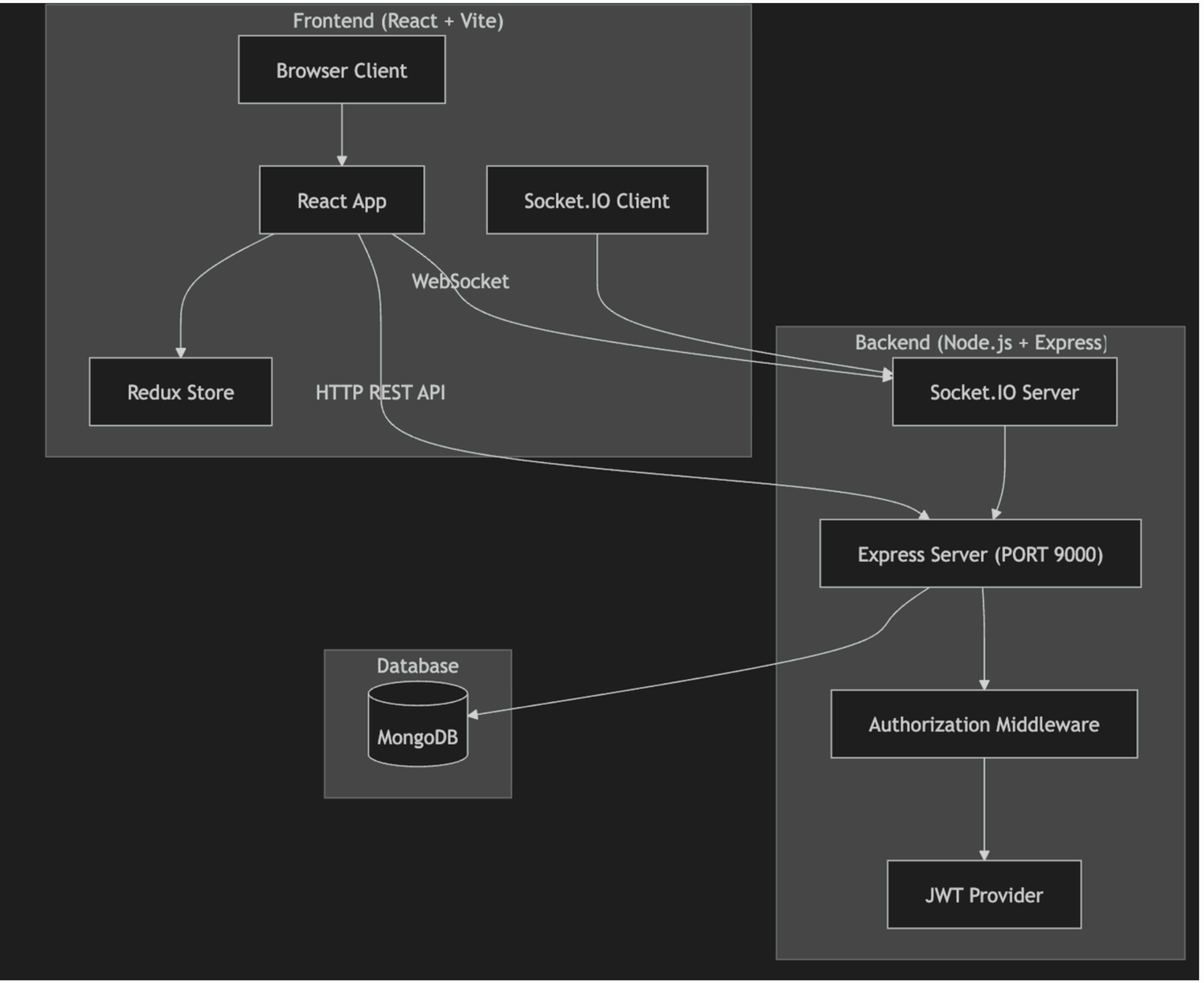
Key Dependencies:

- Controllers depend on Models for database operations.
- Authorization middleware depends on JWTProvider and UserModel.
- Routes use WrapAsync middleware for error handling.

Class Diagram



Chat App:High Level Architecture



API Routes

Method	Endpoint	Auth	Controller	Description
POST	/api/auth/signup	✗	authController.registerUser	Register new user
POST	/api/auth/signin	✗	authController.loginUser	Login user
GET	/api/user/profile	✓	userController.getAuthUser	Get logged-in user profile
GET	/api/user/users	✓	userController.getAllUsers	Get all users except self
POST	/api/chat/	✓	chatController.postChat	Create or get 1-on-1 chat
GET	/api/chat/	✓	chatController.getChat	Get all chats for user
POST	/api/chat/group	✓	chatController.createGroup	Create group chat
DELETE	/api/chat/deleteGroup/:chatId	✓	chatController.deleteGroup	Delete group & its messages
POST	/api/chat/rename	✓	chatController.renameGroup	Rename group
POST	/api/chat/groupremove	✓	chatController.removeFromGroup	Remove user from group
POST	/api/chat/groupadd	✓	chatController.addToGroup	Add user to group
POST	/api/message/	✓	messageController.createMessage	Send a message
GET	/api/message/:chatId	✓	messageController.allMessage	Get all messages in a chat
GET	/api/message/clearChat/:chatId	✓	messageController.clearChat	Clear chat messages

Authentication Flow

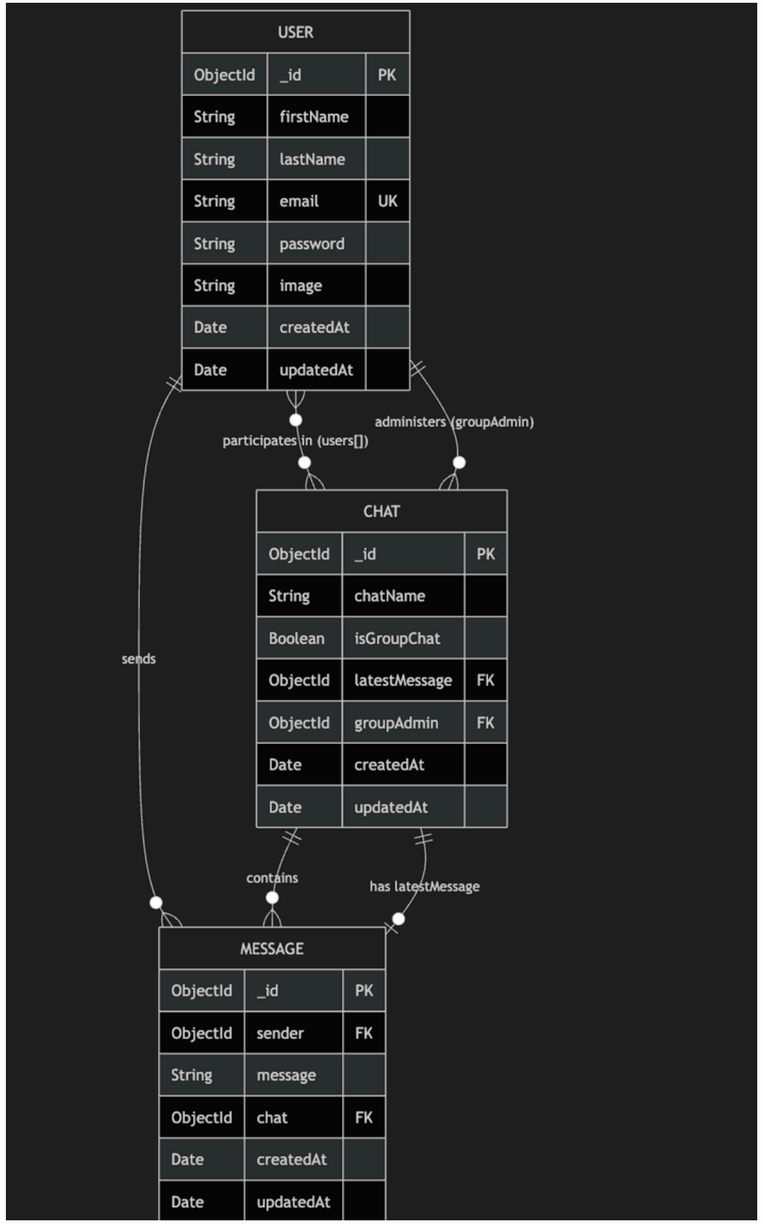
How It Works-

Registration:
1. Client sends `POST /api/auth/signup` with firstName, lastName, email, password.
2. Server checks if user already exists in MongoDB.
3. Password is hashed using `bcryptjs` (8 salt rounds).
4. New user document is saved to the database.
5. A JWT token (48-hour expiry) is generated and returned to the client.

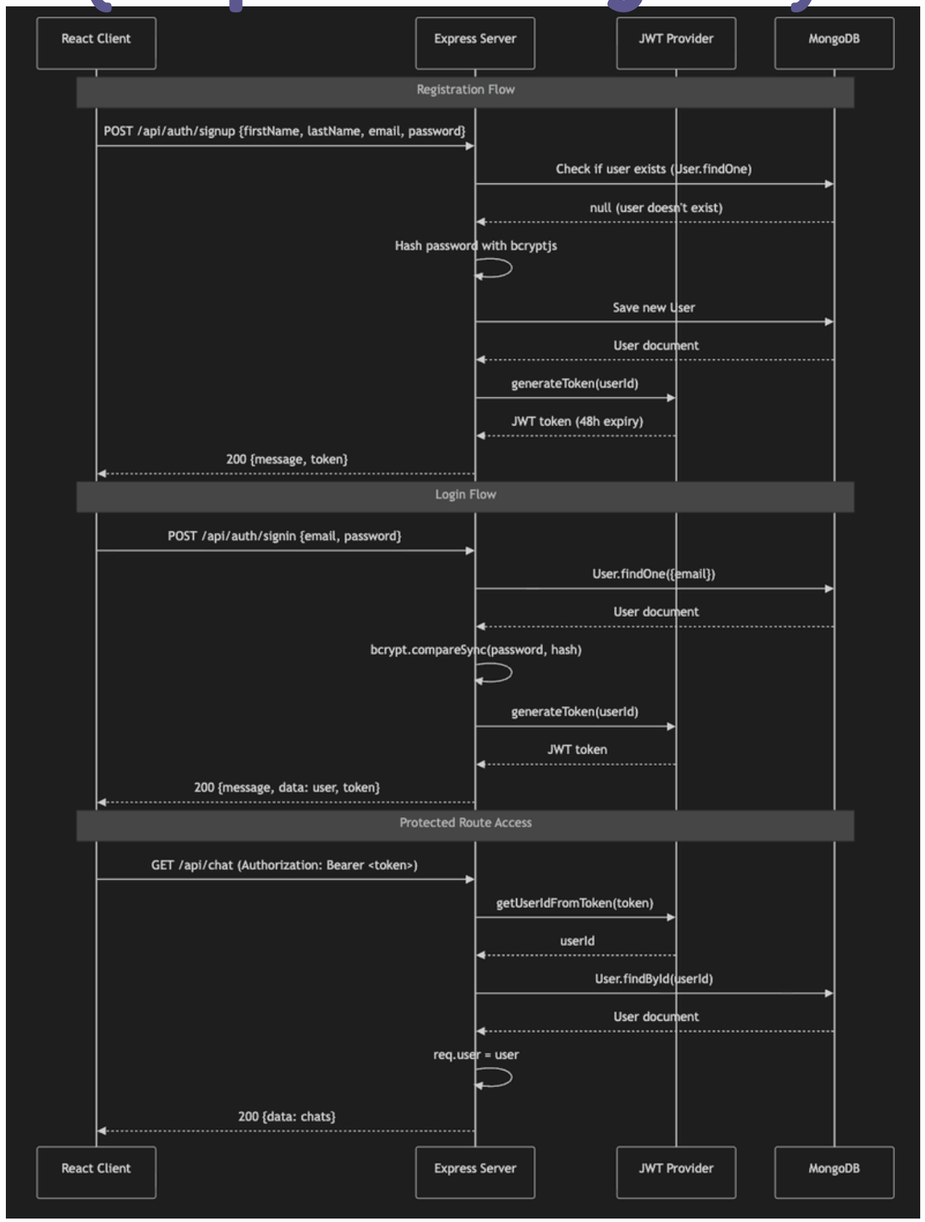
Login:
1. Client sends `POST /api/auth/signin` with email and password.
2. Server finds user by email in MongoDB.
3. Password is verified using `bcrypt.compareSync()`.
4. JWT token is generated and returned along with user data.

Protected Routes:
1. Client includes `Authorization: Bearer <token>` header.
2. Authorization middleware extracts and verifies the token using `jwt.verify()`.
3. User is fetched from DB and attached to `req.user`.
4. Request proceeds to the controller.

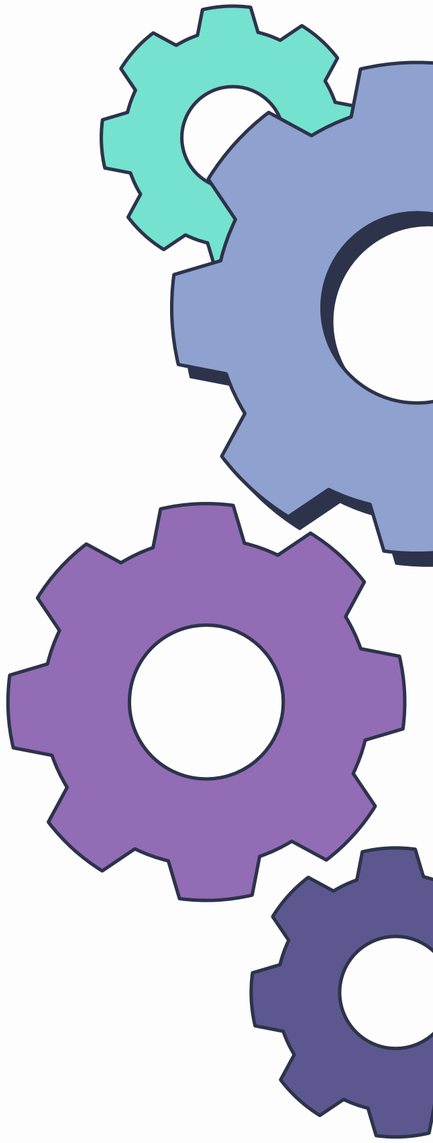
Database:ER Diagram



Authentication Flow (Sequence Diagram)



References :

- Reverse Engineering-
  https://www.geeksforgeeks.org/software-engineering/software-engineering-reverse-engineering/
- Github Repository-
  https://github.com/akashdeep023/Chat_App
- AI Tools-

ChatGPT & Gemini 3.0 Flash